VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

Multidisciplinary Project AI (CO3017)

Report

"Smart AI-IoT Classroom System"

Instructor(s): Võ Thị Ngọc Châu, *CSE - HCMUT*

Students: Lê Ngọc Uy Phong - 235290 *(Group 8)*
Châu Kiến Toàn - 2353192 *(Group 8)*
Nguyễn Duy Thành - 2353101 *(Group 8)*
Nguyễn Tam Long- 2352694 *(Group 8)*

HO CHI MINH CITY, Feb 2026

Contents

List of Figures **5**

List of Tables **5**

Member list & Workload **5**

1 Introduction **6**
1.1 Describe Project . . . 6

2 Requirement Engineering **8**
2.1 Identify Stakeholders . . . 8
2.2 Roles & Expectations . . . 8
2.3 Scope of Project . . . 10
2.4 Functional Requirements . . . 11
2.5 Non-functional Requirements . . . 11
2.6 Requirement Traceability Matrix . . . 12
2.7 Role-Permission Matrix . . . 13
2.8 Summary of Stakeholder Requirements . . . 13

3 System modeling **14**
3.1 Use-case diagram for the whole system . . . 14
3.2 Detail features . . . 15
3.2.1 Feature 1: Campus Hierarchy Monitoring (Monitoring Environment) . . . 15
3.2.2 Feature 2: Room Device Inventory Management (Monitoring Environment) . . . 16
3.2.3 Feature 3: Manual Device Control and Override (Monitoring Environment) . . . 17
3.2.4 Feature 4: IoT Rule Configuration (Monitoring Environment) . . . 18
3.2.5 Feature 5: Session Lifecycle Start (Learning mode) . . . 19
3.2.6 Feature 6: AI Learning Behavior Ingestion and Scoring (Learning mode) . . . 20
3.2.7 Feature 7: Live Learning Analytics Dashboard (Learning mode) . . . 21
3.2.8 Feature 8: Testing Mode Activation and Cheat Detection (Testing mode) . . . 22
3.2.9 Feature 9: Risk Incident Review Workflow (Testing mode) . . . 23
3.2.10 Feature 10: End Session and Finalize Metrics (Testing mode / General) . . . 24

4 Database Design **25**
4.1 Entity types, attributes, relationships . . . 25
4.2 Semantic constraints . . . 25
4.2.1 Entity types constraints . . . 25
4.2.2 Attribute constraints . . . 26
4.2.3 Relationship constraints . . . 26
4.3 EERD Design . . . 27
4.4 Mapping to Database Schema . . . 28
4.4.1 Relational Mapping . . . 28
4.4.2 Mapping Description . . . 28
4.5 Database Schema Implementation . . . 33

5 Database Schema Implementation **33**
5.1 Campus Structure Tables . . . 34
5.1.1 Buildings, Floors, and Rooms . . . 34
5.2 Academic and Session Management Tables . . . 35
5.2.1 Subjects, Teachers, Students, and Enrollments . . . 35
5.2.2 Timetable and Class Sessions . . . 35
5.3 AI Monitoring and Analytics Tables . . . 36
5.3.1 Behavior Classes, Logs, and Aggregates . . . 36
5.4 Risk and Incident Management Tables . . . 37

5.4.1 Risk Behaviors and Incidents . . . 37
5.5 IoT Automation Tables . . . 37
5.5.1 iot_rules and device_states . . . 37
5.6 Scoring Configuration Tables . . . 38
5.6.1 performance_weights and risk_weights . . . 38
5.7 Security and Governance Tables . . . 38
5.7.1 users Table . . . 38
5.7.2 room_occupancy and audit_logs Tables . . . 39
5.8 Indexing Strategy . . . 39
5.9 Implementation Notes . . . 40

6 Hardware Selection **41**
6.1 Processing Unit & Gateway . . . 41
6.2 Input Devices (Sensors & Cameras) . . . 41
6.3 Output Devices (Actuators & Alerts) . . . 41
6.4 Hardware Summary Table . . . 41

7 Device Control Logic **42**
7.1 Lighting Control . . . 42
7.2 HVAC (Fan and AC) Control . . . 42
7.3 Camera and Security Policy . . . 42
7.4 Audio and Visual Control . . . 42

8 Physical Setup and Deployment Layout **43**
8.1 Entrance Zone . . . 43
8.2 Center Wall Zone . . . 43
8.3 Ceiling Zone . . . 43
8.4 Lecturer's Desk (Command Center) . . . 43

9 Dashboard **44**

10 Architecture design **45**

List of Figures

1 System Architecture in Real-World Context 45

List of Tables

1 Member list & workload 5
2 buildings Table 28
3 floors Table 28
4 rooms Table 28
5 subjects Table 28
6 teachers Table 29
7 students Table 29
8 enrollments Table 29
9 timetable Table 29
10 class_sessions Table 30
11 behavior_classes Table 30
12 behavior_logs Table 30
13 performance_aggregates Table 31
14 risk_behaviors Table 31
15 risk_incidents Table 31
16 iot_rules Table 32
17 device_states Table 32
18 performance_weights Table 32
19 risk_weights Table 33
20 users Table 33
21 room_occupancy Table 33
22 audit_logs Table 33
23 Hardware Equipment and Deployment Summary 42

Member list & Workload

No.	Fullname	Student ID	Work	% done
1	Lê Ngọc Uy Phong	2352905		100%
2	Châu Kiến Toàn	2353192		100%
3	Nguyễn Duy Thành	2353101		100%
4	Nguyễn Tam Long	2352694		100%

Table 1: Member list & workload

1 Introduction

Topic: Smart Classroom Automation and Academic Integrity System.
System Name: *Smart Scholar AIoT.*

1.1 Describe Project

Background and motivation: Modern universities operate across many buildings and classrooms, where each room may serve different teaching and examination purposes throughout the day. In such environments, daily operations are often fragmented: room monitoring is handled in one place, device control is handled somewhere else, and examination supervision is managed through separate manual procedures. This creates operational gaps, including delayed responses to classroom issues, inconsistent monitoring quality, and limited traceability when incidents occur. As campus scale increases, these manual and semi-digital workflows become difficult to standardize, audit, and sustain.

Problem statement: Existing classroom operations typically face four major challenges:

- **Operational visibility gap:** Staff cannot quickly obtain a unified live view from building level down to room level.
- **Control inconsistency:** Device management (lights, HVAC-related devices, projectors, fans, cameras) is often ad-hoc, leading to wasted energy and delayed interventions.
- **Monitoring fragmentation:** Teaching analytics and exam-risk supervision are not consistently integrated into a single session lifecycle.
- **Traceability limitations:** Incident handling and review notes are difficult to consolidate for governance and post-session analysis.

Proposed solution: The Smart Scholar AIoT system provides an integrated platform that combines:

- **Campus hierarchy monitoring** (building → floor → room),
- **IoT device inventory and control** (manual operations + automation rule configuration),
- **AI-assisted learning analytics** for classroom sessions,
- **AI-assisted testing supervision** with risk incident creation and review workflow.

Instead of treating these as isolated functions, the system organizes them into one continuous operational lifecycle, from session start to session completion.

System architecture overview: The architecture follows a layered operational model:

- **Classroom/room layer:** each room acts as an operational unit containing device states and session context.
- **Building layer:** rooms are aggregated by floors and buildings for supervisory dashboards.
- **Central service layer:** APIs and database services coordinate session control, analytics, incident management, and rule configuration.

This layered approach supports both local room-level actions (e.g., device overrides, session mode switches) and campus-wide monitoring.

Operational modes and functional perspective: The system supports one always-active operational context and two analytical modes:

- **Monitoring Environment (always-on context):**
  - Navigate campus structure and inspect room operational status.
  - Manage room device inventory and metadata.
  - Apply manual device control and temporary override.

  - Configure IoT rules for occupancy/timetable/idle-based automation.

- **Learning mode (NORMAL session mode):**
  - Start session with room-teacher-subject context.
  - Ingest classroom image frames for behavior detection.
  - Store behavior logs and compute performance-oriented aggregates.
  - Present live analytics and latest annotated frame for instructional awareness.

- **Testing mode (TESTING session mode):**
  - Switch active sessions from NORMAL to TESTING.
  - Detect suspicious behaviors and compute risk scores.
  - Auto-create incidents when risk exceeds threshold.
  - Support incident review, note annotation, and triage tracking.

End-to-end operating flow: A typical operating cycle is structured as follows:

1. Staff selects a target building/room from the hierarchy dashboard.
2. A lecturer starts a classroom session in NORMAL mode.
3. During teaching, frames are processed for learning analytics and engagement insight.
4. If assessment conditions require stricter supervision, the session is switched to TESTING mode.
5. Risk events are detected, stored, and surfaced for review.
6. At session end, final metrics are persisted and the session is closed.

This flow ensures that monitoring, control, analytics, and supervision are coordinated in one coherent process.

Current implementation focus and maturity: The current release emphasizes core operational capabilities that are already available in the system:

- Hierarchical campus monitoring and room dashboards.
- Device inventory CRUD and manual control.
- IoT rule CRUD (configuration layer).
- Session lifecycle control (create, mode switch, analytics, end).
- Learning/testing AI ingestion endpoints with analytics and incident pipelines.
- Incident review workflow with reviewer notes.

Several advanced capabilities are identified as next-phase enhancements, including automatic background rule execution, real-time push alerting, and exportable session reporting.

Project value: The Smart Scholar AIoT system transforms classroom operation from isolated manual tasks into a structured digital workflow. It improves response speed for classroom operations, strengthens consistency in exam supervision, and increases transparency through persistent logs and reviewable incidents. The modular architecture also supports incremental deployment by room, by building, and eventually across the entire university.

2 Requirement Engineering

2.1 Identify Stakeholders

The Smart Scholar AIoT system involves the following primary stakeholders who interact with or rely on the system:

- **Lecturers** – classroom operators who manage sessions and supervise students.
- **Exam Proctors** – responsible for real-time supervision of examinations and handling immediate incidents.
- **Academic Board** – responsible for defining academic policies and reviewing escalated or disputed cases.
- **System Administrators** – deploy, maintain, and secure the platform.
- **Facility Staff** – maintain and manage physical infrastructure and IoT devices (HVAC, lighting, sensors).
- **Cleaning Staff** – responsible for classroom cleaning and allowed to control environmental devices except camera systems.

2.2 Roles & Expectations

Lecturers
Roles

- Deliver lectures and manage classroom teaching activities.
- Operate the system in Learning Mode only.
- Monitor student engagement (e.g., hand-raise events) during lectures.
- Manage classroom flow and interaction during sessions.

Expectations

- Access to a real-time dashboard showing attendance, hand-raise queue, and environmental conditions.
- Simple controls to adjust classroom environment (lighting, HVAC) during teaching.
- Clear visualization of student participation and engagement data.

Exam Proctors
Roles

- Supervise exams in real-time using system dashboards and camera feeds.
- Monitor AI-generated alerts for suspicious behaviors (e.g., cheating detection).
- Take immediate actions during exams (warnings, logging incidents).
- Validate or dismiss AI-detected events during live sessions.

Expectations

- Real-time monitoring interface with live video feeds and alerts.
- Low-latency notifications for suspicious behaviors.
- Clear visualization of detected events (bounding boxes, timestamps).

- Ability to annotate or confirm incidents during the exam.

Academic Board *Roles*

- Define policies and rules for exam monitoring and academic integrity.
- Review flagged incidents after exams for final judgment.
- Handle appeals, disputes, and misconduct investigations.
- Evaluate system effectiveness and adjust policies if needed.

Expectations

- Access to detailed incident reports with evidence (timestamps, video clips).
- Transparent audit trails for all detected and reviewed cases.
- Long-term storage and retrieval of exam records.
- Tools for statistical analysis of cheating patterns and system accuracy.

System Administrators

Roles

- Deploy and maintain servers, edge devices, and IoT nodes.
- Manage user accounts, roles, and security controls.
- Monitor system health and perform backups/patching.

Expectations

- Secure, role-based access control and encrypted storage for sensitive data.
- Monitoring tools, alerting, and runbooks for incidents and recovery.
- Scalable architecture to support multiple classrooms.

Facility Management

Roles

- Maintain physical devices (sensors, relays, HVAC interfaces).
- Respond to hardware faults and schedule preventive maintenance.

Expectations

- Real-time device status and fault notifications.
- Clear mapping between logical controls and physical systems.
- Energy-usage reports to support efficiency measures.

Cleaning Staff

Roles

- Perform cleaning and maintenance tasks inside classrooms.
- Control environmental devices such as lighting, HVAC, and fans during non-class hours.
- Prepare classroom conditions before and after sessions.

Expectations

- Simple interface to control basic classroom devices.
- Restricted access to sensitive systems (no access to cameras or AI monitoring data).
- Clear indication of classroom status (occupied / empty / scheduled).
- Safe and limited permissions to avoid interference with academic activities.

2.3 Scope of Project

Objective

- Automate classroom management using AI and IoT technologies.
- Improve learning comfort and classroom interaction.
- Ensure academic integrity during examinations.
- Provide real-time classroom analytics for lecturers and administrators.
- Integrate with existing university infrastructure.

In-scope

- **Automatic Attendance :** The system shall detect and record student attendance using computer vision with a processing time of less than 1 second per frame and support up to 100 students per classroom.
- **Hand-Raise Detection:** The system shall detect hand-raise events in real time and notify the lecturer within 1 second of detection during Learning Mode.
- **Environmental Monitoring and Control:** The system shall monitor temperature, lighting, and air quality (CO_2 levels) and automatically adjust HVAC, lighting, and fans to maintain predefined comfort thresholds.
- **Energy Efficiency Management:** The system shall automatically turn off lighting, HVAC, and other devices when no occupants are detected in the classroom for more than 5 minutes.
- **Cheat Detection:** During Testing Mode, the system shall detect predefined suspicious behaviors (e.g., looking away from own paper, peer communication, unauthorized device usage) and generate alerts within 2 seconds.
- **Incident Loggin:** The system shall record all detected incidents with timestamps and store at least one image or short video clip (5–10 seconds) for each event.
- **Secure Exam Configuration:** Before exams, the system shall automatically apply predefined configurations, including lighting settings, device restrictions, and network access control.
- **Dashboard System:** The system shall provide:
  - Classroom-level dashboards with real-time updates (delay < 2 seconds)
  - Block-level dashboards aggregating multiple classrooms
  - University-level dashboards for centralized monitoring and reporting
- **Scalability:** The system shall support deployment across multiple classrooms and blocks, with each classroom operating as an independent node connected to a centralized platform.
- **System Availability:** The system shall maintain at least 99% uptime per month.
- **Data Synchronization:** The system shall synchronize classroom data to the central platform within 5 seconds for real-time monitoring and within 24 hours for reporting and analytics.

Out-of-scope

- Replacement of the university's official Learning Management System.
- Development of personal student devices or hardware.
- Full behavioral surveillance outside classroom environments.

2.4 Functional Requirements

Feature 1: Classroom Mode Control

The system shall allow switching between Learning Mode and Testing Mode. Each mode activates a predefined set of functionalities, access permissions, and monitoring policies.

Feature 2: Automatic Attendance

The system shall detect and identify students using computer vision when they enter the classroom and automatically record attendance in the system database.

Feature 3: Hand-Raise Detection

During Learning Mode, the system shall detect hand-raise gestures and notify the lecturer through the dashboard. The system shall log participation events for later review.

Feature 4: Environmental Control

The system shall monitor environmental conditions and automatically control IoT devices (lighting, HVAC, fans) to maintain predefined thresholds. Authorized users (Lecturers, Facility Staff, Cleaning Staff) shall be able to manually adjust environmental settings.

Feature 5: Energy Management

The system shall detect classroom occupancy and automatically turn off devices when the classroom is empty.

Feature 6: Cheat Detection

During Testing Mode, the system shall detect predefined suspicious behaviors and generate real-time alerts for exam proctors.

Feature 7: Incident Logging

The system shall record all detected incidents with timestamps and associated evidence (image or video) and store them for later review.

Feature 8: Secure Exam Configuration

Before an exam session, the system shall automatically configure classroom conditions, including lighting, device restrictions, and network access policies.

Feature 9: Dashboard System

The system shall provide:

- Classroom-level dashboard for lecturers and exam proctors
- Block-level dashboard for facility administrators
- University-level dashboard for centralized monitoring

Feature 10: Role-Based Access Control

The system shall enforce role-based permissions:

- Lecturers: Learning Mode only
- Exam Proctors: Testing Mode monitoring
- Cleaning Staff: control devices except cameras
- System Administrators: full system access

2.5 Non-functional Requirements

Usability

- The system interface shall be simple and task-oriented for each role (Lecturer, Proctor, Cleaning Staff).
- Users shall be able to perform core actions (attendance view, device control, alert handling) within 3 interactions.

Performance

- Real-time events (attendance, hand-raise, alerts) shall be processed and displayed within 2 seconds.

- Dashboard updates shall have a maximum delay of 2 seconds.

Reliability

- The system shall maintain at least 99% availability per month.
- The system shall continue basic operation (attendance, monitoring) in case of temporary network disconnection (edge processing).

Security

- Biometric data (face recognition) shall be encrypted during storage and transmission.
- Role-based access control shall restrict access to sensitive data (e.g., camera feeds).
- Cleaning Staff shall not have access to camera systems or AI monitoring data.

Scalability

- The system shall support deployment across multiple classrooms and buildings.
- Each classroom shall operate independently as a node within the system.

Privacy

- Camera data shall be used only for attendance and exam monitoring purposes.
- Access to recorded data shall be restricted to authorized roles (Proctors, Academic Board).

2.6 Requirement Traceability Matrix

REQ ID	Requirement Description	Mapped Feature	Test Case
REQ-01	Automatic attendance using computer vision	Feature 2: Automatic Attendance	Detect student faces when entering classroom and verify attendance record is created
REQ-02	Classroom environmental control through IoT devices	Feature 4: Environmental Control	Adjust lighting/HVAC and verify system response to sensor data
REQ-03	Hand-raise detection for classroom interaction	Feature 3: Hand-Raise Detection	Simulate hand-raise and verify notification appears on lecturer dashboard
REQ-04	Cheat detection and exam monitoring	Feature 6: Cheat Detection	Simulate suspicious behavior and verify alert is triggered within defined time
REQ-05	Classroom analytics and reporting	Feature 9: Dashboard System	Verify classroom/block/university dashboards display correct aggregated data
REQ-06	Integration with system infrastructure	Feature 10: Role-Based Access Control	Verify different roles access correct system functionalities
REQ-07	Incident recording and audit trail	Feature 7: Incident Logging	Trigger event and verify timestamp + image/video evidence stored correctly
REQ-08	Energy efficiency management	Feature 5: Energy Management	Empty classroom → verify devices are automatically turned off
REQ-09	Secure exam configuration	Feature 8: Secure Exam Configuration	Start exam → verify system applies predefined restrictions

2.7 Role-Permission Matrix

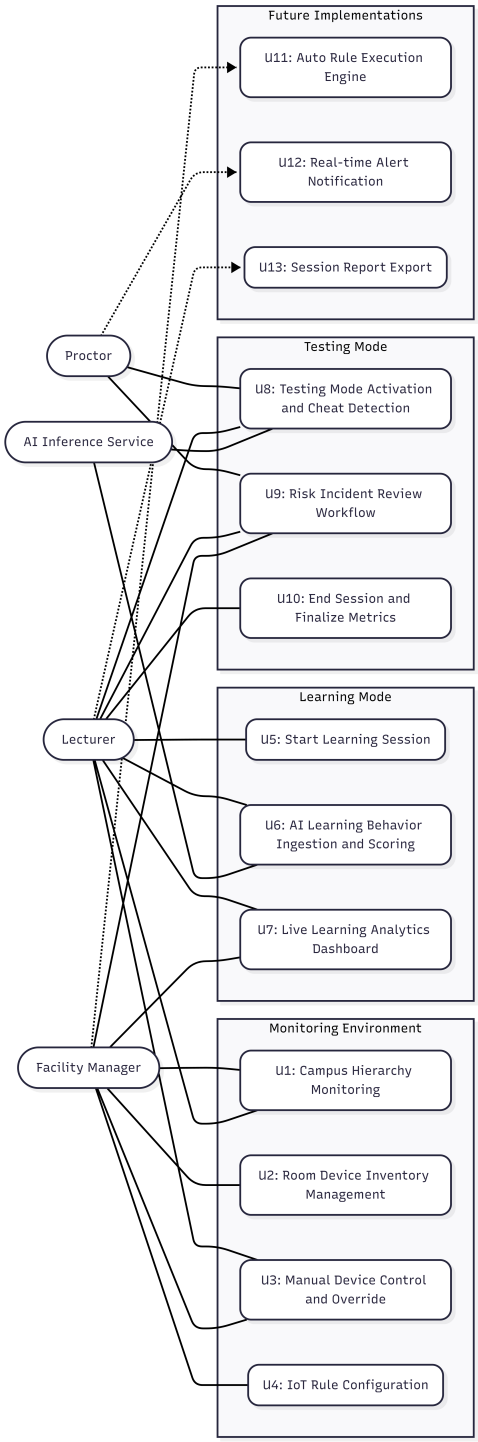
Role	Camera	AI Alerts	Env. Control	Dashboard	Mode
Lecturers	Yes	Yes	Yes	Classroom-level	Learning and Testing
Exam Proctors	Yes	Yes	Yes	Classroom-level	Testing only
Academic Board	Yes (recorded)	Yes	No	University-level	None
System Admin	Yes	Yes	Yes	All levels	None
Facility Staff	No	No	Yes	Block-level	None
Cleaning Staff	No	No	Yes (limited)	Minimal	None

2.8 Summary of Stakeholder Requirements

ID	Requirement Description	Stakeholders
REQ-01	Automatic attendance using computer vision in Learning Mode	Lecturers
REQ-02	Hand-raise detection and student participation tracking	Lecturers
REQ-03	Classroom environmental monitoring and control via IoT devices	Lecturers, Facility Staff, Cleaning Staff
REQ-04	Energy efficiency through automatic device control when classroom is empty	Facility Staff, Cleaning Staff
REQ-05	Cheat detection and real-time exam monitoring in Testing Mode	Exam Proctors
REQ-06	Incident logging with timestamps and evidence for audit purposes	Exam Proctors, Academic Board
REQ-07	Secure exam environment configuration before examinations	Exam Proctors, System Administrators
REQ-08	Multi-level dashboard system (classroom, block, university)	Lecturers, Exam Proctors, Facility Staff, System Administrators
REQ-09	Role-based access control (RBAC) for all system functionalities	System Administrators
REQ-10	Post-exam review, reporting, and decision-making tools	Academic Board
REQ-11	System deployment, maintenance, and infrastructure management	System Administrators

3 System modeling

3.1 Use-case diagram for the whole system



3.2 Detail features

3.2.1 Feature 1: Campus Hierarchy Monitoring (Monitoring Environment)

User story

As a Facility Manager or Lecturer, I want to browse building groups, buildings, floors, and rooms with live counts, so that I can quickly locate where sessions are active and where rooms are online.

Use-case using a table format

Use case ID:	U1
Use case name:	Campus Hierarchy Monitoring
Created by:	System Modeling Draft
Last updated by:	System Modeling Draft
Date created:	21/03/2026
Date last updated:	21/03/2026
Actors:	Facility Manager, Lecturer
Description:	View building groups and navigate to building/floor/room levels with operational indicators (active sessions, rooms online).
Trigger:	User opens the system dashboard.
Preconditions:	System data for buildings, floors, and rooms exists.
Postconditions:	User reaches selected room context for further monitoring/control tasks.
Normal Flows:	1. User opens overview page. 2. System shows group-level metrics. 3. User selects a group and then a building. 4. System loads floors and rooms. 5. User selects a room for detailed monitoring.
Alternative Flows:	A1: User searches by building code/name/location to filter faster.
Exceptions:	E1: Data load fails → show error and allow retry.
Note and issues:	Implemented in current UI/API.

3.2.2 Feature 2: Room Device Inventory Management (Monitoring Environment)

User story

As a Facility Manager, I want to add, edit, and remove room devices with position metadata, so that device inventory matches the real classroom setup.

Use-case using a table format

Use case ID:	U2
Use case name:	Room Device Inventory Management
Created by:	System Modeling Draft
Last updated by:	System Modeling Draft
Date created:	21/03/2026
Date last updated:	21/03/2026
Actors:	Facility Manager
Description:	Manage room device list (create/update/delete) with type, location, and power attributes.
Trigger:	User opens room devices panel and clicks add/edit/delete.
Preconditions:	Room exists and user has management rights.
Postconditions:	Device inventory and state tracking are updated consistently.
Normal Flows:	1. User selects a room. 2. System displays current devices. 3. User adds or edits metadata (type/location/power). 4. User confirms changes. 5. System saves and refreshes list.
Alternative Flows:	A1: User removes an obsolete device entry.
Exceptions:	E1: Duplicate device ID → reject creation. E2: Invalid location format → validation error.
Note and issues:	Implemented in current backend and dashboard.

3.2.3 Feature 3: Manual Device Control and Override (Monitoring Environment)

User story

As a Lecturer, I want to manually toggle classroom devices and optionally set temporary override duration, so that I can adapt room conditions during class.

Use-case using a table format

Use case ID:	U3
Use case name:	Manual Device Control and Override
Created by:	System Modeling Draft
Last updated by:	System Modeling Draft
Date created:	21/03/2026
Date last updated:	21/03/2026
Actors:	Lecturer, Facility Manager
Description:	Toggle ON/OFF state for a specific device and set/clear manual override to temporarily bypass automation.
Trigger:	User clicks a device control action in room dashboard.
Preconditions:	Target device exists in selected room.
Postconditions:	Device status and override flags are updated.
Normal Flows:	1. User picks room and device. 2. User selects action ON/OFF. 3. User optionally sets override duration. 4. System updates state and returns timestamp.
Alternative Flows:	A1: User clears override to return to auto mode.
Exceptions:	E1: Device not found → operation rejected.
Note and issues:	Implemented. Role restriction can be strengthened later.

3.2.4 Feature 4: IoT Rule Configuration (Monitoring Environment)

User story

As a Facility Manager, I want to create and maintain occupancy/timetable/idle rules, so that classroom devices can be controlled automatically by policy.

Use-case using a table format

Use case ID:	U4
Use case name:	IoT Rule Configuration
Created by:	System Modeling Draft
Last updated by:	System Modeling Draft
Date created:	21/03/2026
Date last updated:	21/03/2026
Actors:	Facility Manager
Description:	Create, update, activate/deactivate, and delete IoT rules for room-level automation.
Trigger:	User opens rules management and submits a rule form.
Preconditions:	Room exists; rule condition type is valid.
Postconditions:	Rule is persisted with priority and active state.
Normal Flows:	1. User selects room. 2. User enters condition type and parameters. 3. User defines device actions and priority. 4. System validates and saves rule.
Alternative Flows:	A1: User uses occupancy/zero-occupancy template helpers.
Exceptions:	E1: Invalid condition type → reject request.
Note and issues:	CRUD is implemented; automatic execution engine is not fully implemented yet.

3.2.5 Feature 5: Session Lifecycle Start (Learning mode)

User story

As a Lecturer, I want to start a classroom session with room, teacher, subject, and student list context, so that AI monitoring is attached to the correct class session.

Use-case using a table format

Use case ID:	U5
Use case name:	Start Learning Session
Created by:	System Modeling Draft
Last updated by:	System Modeling Draft
Date created:	21/03/2026
Date last updated:	21/03/2026
Actors:	Lecturer
Description:	Create an ACTIVE session in NORMAL mode with room, teacher, and subject bindings.
Trigger:	Lecturer initiates class session start.
Preconditions:	Referenced room, teacher, and subject are valid.
Postconditions:	Session is created and ready for learning/test ingestion.
Normal Flows:	1. Lecturer submits session metadata. 2. System validates references. 3. System creates ACTIVE session in NORMAL mode.
Alternative Flows:	A1: Students list may be omitted and added progressively.
Exceptions:	E1: Invalid room/teacher/subject → reject creation.
Note and issues:	Implemented.

3.2.6 Feature 6: AI Learning Behavior Ingestion and Scoring (Learning mode)

User story

As a Lecturer, I want the system to process classroom image frames in learning mode and compute student performance, so that engagement can be measured continuously.

Use-case using a table format

Use case ID:	U6
Use case name:	AI Learning Behavior Ingestion and Scoring
Created by:	System Modeling Draft
Last updated by:	System Modeling Draft
Date created:	21/03/2026
Date last updated:	21/03/2026
Actors:	Lecturer, AI Inference Service
Description:	Process incoming frame, detect learning behaviors, store logs, and update per-student performance aggregates.
Trigger:	Client sends frame to learning endpoint.
Preconditions:	Session is ACTIVE; AI model is available.
Postconditions:	Detection logs and scores are updated; annotated frame returned.
Normal Flows:	1. System receives frame with confidence threshold. 2. YOLO inference runs in LEARNING mode. 3. Detections are stored in behavior logs. 4. Scoring engine calculates and updates performance aggregates. 5. System returns detections and annotated image.
Alternative Flows:	A1: Student ID can be provided or inferred by detection pipeline.
Exceptions:	E1: YOLO unavailable → service unavailable response. E2: Invalid image payload → reject request.
Note and issues:	Implemented; behavior coverage depends on trained model classes.

3.2.7 Feature 7: Live Learning Analytics Dashboard (Learning mode)

User story

As a Lecturer, I want to view live analytics (behavior counts, elapsed time, activity trends), so that I can adapt teaching actions during class.

Use-case using a table format

Use case ID:	U7
Use case name:	Live Learning Analytics Dashboard
Created by:	System Modeling Draft
Last updated by:	System Modeling Draft
Date created:	21/03/2026
Date last updated:	21/03/2026
Actors:	Lecturer, Facility Manager
Description:	Retrieve session analytics and latest annotated frame for real-time dashboard insight.
Trigger:	Dashboard refreshes analytics for selected session.
Preconditions:	Session exists.
Postconditions:	User sees updated student/teacher behavior metrics and frame preview.
Normal Flows:	1. User selects active session. 2. System returns analytics summary. 3. System returns latest frame (live or incident fallback). 4. Dashboard renders trends and top behaviors.
Alternative Flows:	A1: If no frame exists yet, dashboard shows empty state.
Exceptions:	E1: Session not found → return error state.
Note and issues:	Implemented.

3.2.8 Feature 8: Testing Mode Activation and Cheat Detection (Testing mode)

User story

As a Proctor/Lecturer, I want to switch an active session to testing mode and run suspicious-behavior detection, so that potential cheating can be identified in near real time.

Use-case using a table format

Use case ID:	U8
Use case name:	Testing Mode Activation and Cheat Detection
Created by:	System Modeling Draft
Last updated by:	System Modeling Draft
Date created:	21/03/2026
Date last updated:	21/03/2026
Actors:	Proctor/Lecturer, AI Inference Service
Description:	Switch session mode to TESTING and process testing frames for risk analysis.
Trigger:	User changes mode and starts test-frame ingestion.
Preconditions:	Session is ACTIVE; mode is TESTING before test ingestion.
Postconditions:	Risk analysis is returned; high-risk students may be auto-flagged.
Normal Flows:	1. User requests mode change to TESTING. 2. System validates active session and updates mode. 3. Client sends frame to testing endpoint. 4. YOLO detects suspicious behaviors and risk detector scores each student. 5. System returns risk analysis and incident IDs created.
Alternative Flows:	A1: Session may switch back to NORMAL when exam ends.
Exceptions:	E1: Session not ACTIVE or mode invalid → reject change. E2: Session not in TESTING for test endpoint → reject ingestion.
Note and issues:	Implemented.

3.2.9 Feature 9: Risk Incident Review Workflow (Testing mode)

User story

As a Proctor/Lecturer, I want to view incidents, inspect details/snapshots, and add review notes, so that suspicious events are triaged and documented.

Use-case using a table format

Use case ID:	U9
Use case name:	Risk Incident Review Workflow
Created by:	System Modeling Draft
Last updated by:	System Modeling Draft
Date created:	21/03/2026
Date last updated:	21/03/2026
Actors:	Proctor/Lecturer, Facility Manager
Description:	List incidents by room/session, review unreviewed items, and attach reviewer notes.
Trigger:	User opens incident panel and selects an incident.
Preconditions:	Incident records exist for selected scope.
Postconditions:	Incident is marked reviewed and notes are stored.
Normal Flows:	1. User filters incidents by room/session/reviewed state. 2. User opens incident detail. 3. User checks risk score, behaviors, and snapshot. 4. User submits reviewer notes. 5. System marks incident reviewed.
Alternative Flows:	A1: User fetches only unreviewed incidents for fast triage.
Exceptions:	E1: Incident not found → show error. E2: Snapshot missing → show detail without image.
Note and issues:	Implemented.

3.2.10 Feature 10: End Session and Finalize Metrics (Testing mode / General)

User story

As a Lecturer, I want to end a running session and finalize computed scores, so that class records are closed and ready for later analysis.

Use-case using a table format

Use case ID:	U10
Use case name:	End Session and Finalize Metrics
Created by:	System Modeling Draft
Last updated by:	System Modeling Draft
Date created:	21/03/2026
Date last updated:	21/03/2026
Actors:	Lecturer
Description:	Close an ACTIVE session, stamp end time, and compute final aggregate performance data.
Trigger:	Lecturer clicks End Session.
Preconditions:	Session is ACTIVE.
Postconditions:	Session status becomes COMPLETED with end timestamp and duration.
Normal Flows:	1. Lecturer selects End Session. 2. System validates session state. 3. System computes final scores from behavior logs. 4. System stores aggregates and returns summary.
Alternative Flows:	A1: User can still query completed session analytics/history later.
Exceptions:	E1: Session already closed → reject end action.
Note and issues:	Implemented.

4 Database Design

4.1 Entity types, attributes, relationships

The system includes the following main entities and relationships:

At the infrastructure core are **Building**, **Floor**, and **Room**. A Building has many Floors (1:N), a Floor has many Rooms (1:N), and each Room belongs to exactly one Floor. Room stores code, capacity, and a flexible device inventory structure.

Academic entities include **Subject**, **Teacher**, **Student**, and **Enrollment**. Student and Subject are connected via Enrollment in an N:M relationship. A Subject can have many sessions; a Teacher can teach many sessions.

Operational class runtime is represented by **ClassSession**, optionally linked to **Timetable**. Each ClassSession belongs to exactly one Room and may reference one Teacher and one Subject. Session stores mode (NORMAL/TESTING), status (ACTIVE/COMPLETED/CANCELLED), start/end timestamps, and student-present list.

Behavior analytics entities include **BehaviorClass**, **BehaviorLog**, and **PerformanceAggregate**. BehaviorClass defines possible labels and actor type. BehaviorLog records detected behaviors per session and actor with confidence and optional frame snapshot. PerformanceAggregate stores computed score summaries and behavior breakdown per actor/session.

Risk subsystem entities include **RiskBehavior**, **RiskIncident**, and **RiskWeight**. RiskIncident belongs to exactly one ClassSession and one Student (N:1 to each), storing risk score, risk level, triggered behaviors, and review state. RiskWeight keeps threshold/weight parameters used for risk scoring.

IoT subsystem entities include **IoTRule**, **DeviceState**, and **RoomOccupancy**. A Room has many IoTRules (1:N) and many DeviceState rows (1:N). RoomOccupancy is modeled as 1:1 with Room. IoTRule stores condition type, condition parameters, action list, priority, and active state.

Configuration and security entities include **PerformanceWeight**, **User**, and **AuditLog**. PerformanceWeight defines behavior weights globally or by subject. User supports authentication/authorization (Admin, Lecturer, Facility Manager). AuditLog records actions over entities for traceability.

4.2 Semantic constraints

The system enforces the following business rules for integrity and operational consistency:

4.2.1 Entity types constraints

- **Campus hierarchy:**
  - Building names are unique.
  - Floor numbering is unique within each building.
  - Room codes are unique globally.
  - Room capacity must satisfy capacity > 0.
- **Academic core:**
  - Student student_id is unique.
  - Subject code is unique.
  - Enrollment must be unique for each pair $(\text{student_id}, \text{subject_id})$.
- **Session management:**
  - Session mode domain is $\{\text{NORMAL}, \text{TESTING}\}$.
  - Session status domain is $\{\text{ACTIVE}, \text{COMPLETED}, \text{CANCELLED}\}$.
  - If status is COMPLETED, then end_time must be non-null and $end_time \geq start_time$.
- **Identity & access:**

- Username is unique in User.
- Email is unique when provided.
- Role domain is $\{\text{ADMIN}, \text{LECTURER}, \text{FACILITY_MANAGER}\}$.
- Only active users ($is_active = true$) can authenticate.

4.2.2 Attribute constraints

- **Behavior & scoring:**
  - Detection confidence must satisfy $0 \leq \text{yolo_confidence} \leq 1$.
  - Behavior count must satisfy $\text{count} \geq 1$.
  - Duration must satisfy $\text{duration_seconds} \geq 0$.
  - Performance score is normalized to $0 \leq \text{score} \leq 100$.
- **Risk subsystem:**
  - Risk score must satisfy $0 \leq \text{risk_score} \leq 1$ (or mapped equivalent scale).
  - Risk level domain is $\{\text{LOW}, \text{MEDIUM}, \text{HIGH}, \text{CRITICAL}\}$.
  - Reviewed incidents require reviewed_at timestamp and optional reviewer notes.
- **IoT subsystem:**
  - DeviceState status domain is $\{\text{ON}, \text{OFF}, \text{ERROR}\}$.
  - Manual override expiration, when present, must satisfy $override_until \geq now()$ at set time.
  - Rule condition_type domain is $\{\text{OCCUPANCY}, \text{TIMETABLE}, \text{ZERO_OCCUPANCY}, \text{TIME_BASED}\}$.
- **Occupancy tracking:**
  - Occupancy count must satisfy $\text{occupancy_count} \geq 0$.
  - Boolean occupancy flag should be consistent with count:

$$\text{is_occupied} = (\text{occupancy_count} > 0)$$

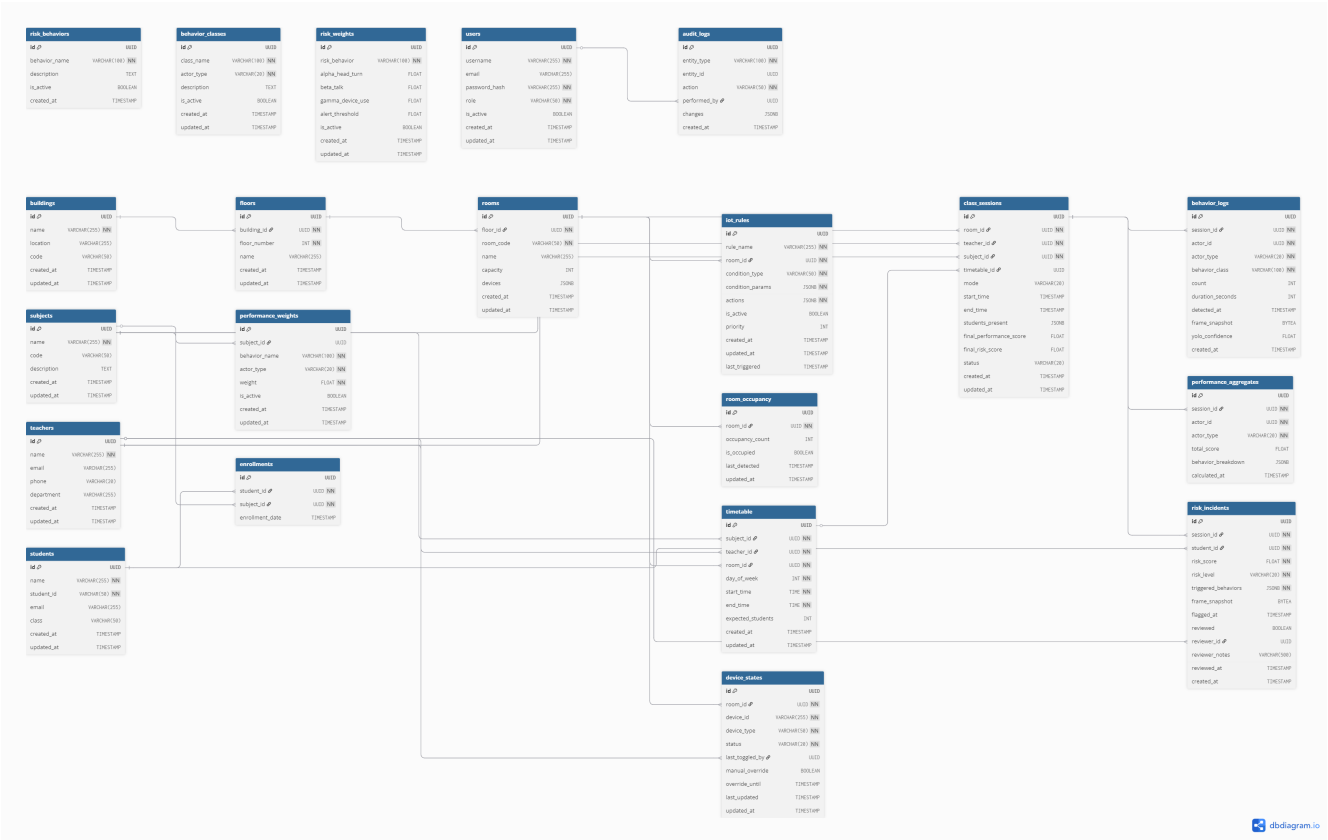
4.2.3 Relationship constraints

- **Building – Floor – Room:**
  - Total participation for Floor in Building and Room in Floor.
  - Deleting a Building deletes dependent Floors/Rooms (cascade policy).
- **ClassSession – Room/Teacher/Subject:**
  - Every session must reference exactly one Room.
  - Teacher and Subject references must exist when assigned.
  - A session in ACTIVE state should be unique per room-time context (application-level scheduling rule).
- **ClassSession – BehaviorLog – PerformanceAggregate:**
  - BehaviorLog cannot exist without a valid ClassSession.
  - PerformanceAggregate is unique per $(\text{session_id}, \text{actor_id}, \text{actor_type})$.
- **ClassSession – RiskIncident – Student:**
  - RiskIncident must reference exactly one valid session and one valid student.

  - Incident review should be performed by authorized reviewer roles only (application-layer RBAC).

- **Room – DeviceState/IoTRule/RoomOccupancy:**
  - DeviceState is unique by $(room_id, device_id)$.
  - RoomOccupancy has total participation on room side for monitored rooms (recommended operational rule).
  - Rule execution must respect manual override precedence: auto-actions cannot overwrite manually overridden devices until override expires or is cleared.

4.3 EERD Design



4.4 Mapping to Database Schema

4.4.1 Relational Mapping

4.4.2 Mapping Description

Attribute	Data Type	Constraints
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()
name	VARCHAR(255)	NOT NULL, UNIQUE
location	VARCHAR(255)	NULL
code	VARCHAR(50)	NULL
created_at	TIMESTAMP	DEFAULT NOW()
updated_at	TIMESTAMP	DEFAULT NOW()

Table 2: buildings Table

Attribute	Data Type	Constraints
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()
building_id	UUID	FK REFERENCES buildings(id), NOT NULL, ON DELETE CASCADE
floor_number	INT	NOT NULL
name	VARCHAR(255)	NULL
created_at	TIMESTAMP	DEFAULT NOW()
updated_at	TIMESTAMP	DEFAULT NOW()
Composite Unique		UNIQUE(building_id, floor_number)

Table 3: floors Table

Attribute	Data Type	Constraints
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()
floor_id	UUID	FK REFERENCES floors(id), NOT NULL, ON DELETE CASCADE
room_code	VARCHAR(50)	NOT NULL, UNIQUE
name	VARCHAR(255)	NULL
capacity	INT	DEFAULT 30
devices	JSONB	DEFAULT '{"device_list": []}'
created_at	TIMESTAMP	DEFAULT NOW()
updated_at	TIMESTAMP	DEFAULT NOW()

Table 4: rooms Table

Attribute	Data Type	Constraints
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()
name	VARCHAR(255)	NOT NULL, UNIQUE
code	VARCHAR(50)	UNIQUE
description	TEXT	NULL
created_at	TIMESTAMP	DEFAULT NOW()
updated_at	TIMESTAMP	DEFAULT NOW()

Table 5: subjects Table

Attribute	Data Type	Constraints
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()
name	VARCHAR(255)	NOT NULL
email	VARCHAR(255)	UNIQUE
phone	VARCHAR(20)	NULL
department	VARCHAR(255)	NULL
created_at	TIMESTAMP	DEFAULT NOW()
updated_at	TIMESTAMP	DEFAULT NOW()

Table 6: teachers Table

Attribute	Data Type	Constraints
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()
name	VARCHAR(255)	NOT NULL
student_id	VARCHAR(50)	NOT NULL, UNIQUE
email	VARCHAR(255)	UNIQUE
class	VARCHAR(50)	NULL
created_at	TIMESTAMP	DEFAULT NOW()
updated_at	TIMESTAMP	DEFAULT NOW()

Table 7: students Table

Attribute	Data Type	Constraints
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()
student_id	UUID	FK REFERENCES students(id), NOT NULL, ON DELETE CASCADE
subject_id	UUID	FK REFERENCES subjects(id), NOT NULL, ON DELETE CASCADE
enrollment_date	TIMESTAMP	DEFAULT NOW()
Composite Unique		UNIQUE(student_id, subject_id)

Table 8: enrollments Table

Attribute	Data Type	Constraints
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()
subject_id	UUID	FK REFERENCES subjects(id), NOT NULL, ON DELETE CASCADE
teacher_id	UUID	FK REFERENCES teachers(id), NOT NULL, ON DELETE CASCADE
room_id	UUID	FK REFERENCES rooms(id), NOT NULL, ON DELETE CASCADE
day_of_week	INT	NOT NULL
start_time	TIME	NOT NULL
end_time	TIME	NOT NULL
expected_students	INT	DEFAULT 0
created_at	TIMESTAMP	DEFAULT NOW()
updated_at	TIMESTAMP	DEFAULT NOW()

Table 9: timetable Table

Attribute	Data Type	Constraints
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()
room_id	UUID	FK REFERENCES rooms(id), NOT NULL, ON DELETE CASCADE
teacher_id	UUID	FK REFERENCES teachers(id), NOT NULL
subject_id	UUID	FK REFERENCES subjects(id), NOT NULL
timetable_id	UUID	FK REFERENCES timetable(id), NULL
mode	VARCHAR(20)	DEFAULT 'NORMAL'
start_time	TIMESTAMP	DEFAULT NOW()
end_time	TIMESTAMP	NULL
students_present	JSONB	DEFAULT '[]'
final_performance_score	FLOAT	NULL
final_risk_score	FLOAT	NULL
status	VARCHAR(20)	DEFAULT 'ACTIVE'
created_at	TIMESTAMP	DEFAULT NOW()
updated_at	TIMESTAMP	DEFAULT NOW()

Table 10: class_sessions Table

Attribute	Data Type	Constraints
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()
class_name	VARCHAR(100)	NOT NULL, UNIQUE
actor_type	VARCHAR(20)	NOT NULL
description	TEXT	NULL
is_active	BOOLEAN	DEFAULT TRUE
created_at	TIMESTAMP	DEFAULT NOW()
updated_at	TIMESTAMP	DEFAULT NOW()

Table 11: behavior_classes Table

Attribute	Data Type	Constraints
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()
session_id	UUID	FK REFERENCES class_sessions(id), NOT NULL, ON DELETE CASCADE
actor_id	UUID	NOT NULL
actor_type	VARCHAR(20)	NOT NULL
behavior_class	VARCHAR(100)	NOT NULL
count	INT	DEFAULT 1
duration_seconds	INT	DEFAULT 0
detected_at	TIMESTAMP	DEFAULT NOW()
frame_snapshot	BYTEA	NULL
yolo_confidence	FLOAT	DEFAULT 0.0
created_at	TIMESTAMP	DEFAULT NOW()

Table 12: behavior_logs Table

Attribute	Data Type	Constraints
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()
session_id	UUID	FK REFERENCES class_sessions(id), NOT NULL, ON DELETE CASCADE
actor_id	UUID	NOT NULL
actor_type	VARCHAR(20)	NOT NULL
total_score	FLOAT	DEFAULT 0.0
behavior_breakdown	JSONB	DEFAULT '{}'
calculated_at	TIMESTAMP	DEFAULT NOW()
Composite Unique		UNIQUE(session_id, actor_id)

Table 13: performance_aggregates Table

Attribute	Data Type	Constraints
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()
behavior_name	VARCHAR(100)	NOT NULL, UNIQUE
description	TEXT	NULL
is_active	BOOLEAN	DEFAULT TRUE
created_at	TIMESTAMP	DEFAULT NOW()

Table 14: risk_behaviors Table

Attribute	Data Type	Constraints
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()
session_id	UUID	FK REFERENCES class_sessions(id), NOT NULL, ON DELETE CASCADE
student_id	UUID	FK REFERENCES students(id), NOT NULL
risk_score	FLOAT	NOT NULL
risk_level	VARCHAR(20)	NOT NULL
triggered_behaviors	JSONB	NOT NULL
frame_snapshot	BYTEA	NULL
flagged_at	TIMESTAMP	DEFAULT NOW()
reviewed	BOOLEAN	DEFAULT FALSE
reviewer_id	UUID	FK REFERENCES teachers(id), NULL
reviewer_notes	VARCHAR(500)	NULL
reviewed_at	TIMESTAMP	NULL
created_at	TIMESTAMP	DEFAULT NOW()

Table 15: risk_incidents Table

Attribute	Data Type	Constraints
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()
rule_name	VARCHAR(255)	NOT NULL
room_id	UUID	FK REFERENCES rooms(id), NOT NULL, ON DELETE CASCADE
condition_type	VARCHAR(50)	NOT NULL
condition_params	JSONB	NOT NULL
actions	JSONB	NOT NULL
is_active	BOOLEAN	DEFAULT TRUE
priority	INT	DEFAULT 0
created_at	TIMESTAMP	DEFAULT NOW()
updated_at	TIMESTAMP	DEFAULT NOW()
last_triggered	TIMESTAMP	NULL

Table 16: iot_rules Table

Attribute	Data Type	Constraints
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()
room_id	UUID	FK REFERENCES rooms(id), NOT NULL, ON DELETE CASCADE
device_id	VARCHAR(255)	NOT NULL
device_type	VARCHAR(50)	NOT NULL
status	VARCHAR(20)	NOT NULL, DEFAULT 'OFF'
last_toggled_by	UUID	FK REFERENCES teachers(id), NULL
manual_override	BOOLEAN	DEFAULT FALSE
override_until	TIMESTAMP	NULL
last_updated	TIMESTAMP	DEFAULT NOW()
updated_at	TIMESTAMP	DEFAULT NOW()
Composite Unique		UNIQUE(room_id, device_id)

Table 17: device_states Table

Attribute	Data Type	Constraints
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()
subject_id	UUID	FK REFERENCES subjects(id), NULL, ON DELETE SET NULL
behavior_name	VARCHAR(100)	NOT NULL
actor_type	VARCHAR(20)	NOT NULL
weight	FLOAT	NOT NULL
is_active	BOOLEAN	DEFAULT TRUE
created_at	TIMESTAMP	DEFAULT NOW()
updated_at	TIMESTAMP	DEFAULT NOW()
Composite Unique		UNIQUE(subject_id, behavior_name, actor_type)

Table 18: performance_weights Table

Attribute	Data Type	Constraints
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()
risk_behavior	VARCHAR(100)	NOT NULL, UNIQUE
alpha_head_turn	FLOAT	DEFAULT 0.3
beta_talk	FLOAT	DEFAULT 0.5
gamma_device_use	FLOAT	DEFAULT 0.8
alert_threshold	FLOAT	DEFAULT 50.0
is_active	BOOLEAN	DEFAULT TRUE
created_at	TIMESTAMP	DEFAULT NOW()
updated_at	TIMESTAMP	DEFAULT NOW()

Table 19: risk_weights Table

Attribute	Data Type	Constraints
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()
username	VARCHAR(255)	NOT NULL, UNIQUE
email	VARCHAR(255)	UNIQUE
password_hash	VARCHAR(255)	NOT NULL
role	VARCHAR(50)	NOT NULL, DEFAULT 'LECTURER'
is_active	BOOLEAN	DEFAULT TRUE
created_at	TIMESTAMP	DEFAULT NOW()
updated_at	TIMESTAMP	DEFAULT NOW()

Table 20: users Table

Attribute	Data Type	Constraints
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()
room_id	UUID	FK REFERENCES rooms(id), NOT NULL, UNIQUE, ON DELETE CASCADE
occupancy_count	INT	DEFAULT 0
is_occupied	BOOLEAN	DEFAULT FALSE
last_detected	TIMESTAMP	DEFAULT NOW()
updated_at	TIMESTAMP	DEFAULT NOW()

Table 21: room_occupancy Table

Attribute	Data Type	Constraints
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()
entity_type	VARCHAR(100)	NOT NULL
entity_id	UUID	NULL
action	VARCHAR(50)	NOT NULL
performed_by	UUID	FK REFERENCES users(id), NULL
changes	JSONB	NULL
created_at	TIMESTAMP	DEFAULT NOW()

Table 22: audit_logs Table

4.5 Database Schema Implementation

5 Database Schema Implementation

The Smart AI-IoT Classroom database consists of the following table categories:

- **Campus Structure:** buildings, floors, rooms (3 tables)
- **Academic & Session Management:** subjects, teachers, students, enrollments, timetable, class_sessions (6 tables)
- **AI Monitoring & Analytics:** behavior_classes, behavior_logs, performance_aggregates (3 tables)
- **Risk & Incident Management:** risk_behaviors, risk_incidents (2 tables)
- **IoT Automation:** iot_rules, device_states (2 tables)
- **Scoring Configuration:** performance_weights, risk_weights (2 tables)
- **Security & Governance:** users, room_occupancy, audit_logs (3 tables)

5.1 Campus Structure Tables

5.1.1 Buildings, Floors, and Rooms

These tables implement the campus hierarchy Building → Floor → Room with cascade deletion from parent to child.

```sql
CREATE TABLE IF NOT EXISTS buildings (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name VARCHAR(255) NOT NULL UNIQUE,
  location VARCHAR(255),
  code VARCHAR(50),
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS floors (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  building_id UUID NOT NULL REFERENCES buildings(id) ON DELETE CASCADE,
  floor_number INT NOT NULL,
  name VARCHAR(255),
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW(),
  UNIQUE(building_id, floor_number)
);

CREATE TABLE IF NOT EXISTS rooms (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  floor_id UUID NOT NULL REFERENCES floors(id) ON DELETE CASCADE,
  room_code VARCHAR(50) NOT NULL UNIQUE,
  name VARCHAR(255),
  capacity INT DEFAULT 30,
  devices JSONB DEFAULT '{"device_list": []}',
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
);
```

Implement 1: Campus structure tables

Key Features:

- Hierarchical integrity via foreign keys with ON DELETE CASCADE.
- Room device inventory is stored as JSONB for flexible schema evolution.
- Unique constraints prevent duplicate floor numbers in the same building and duplicate room codes globally.

5.2 Academic and Session Management Tables

5.2.1 Subjects, Teachers, Students, and Enrollments

These tables model academic actors and many-to-many enrollment between students and subjects.

```sql
CREATE TABLE IF NOT EXISTS subjects (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name VARCHAR(255) NOT NULL UNIQUE,
  code VARCHAR(50) UNIQUE,
  description TEXT,
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS teachers (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name VARCHAR(255) NOT NULL,
  email VARCHAR(255) UNIQUE,
  phone VARCHAR(20),
  department VARCHAR(255),
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS students (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name VARCHAR(255) NOT NULL,
  student_id VARCHAR(50) UNIQUE NOT NULL,
  email VARCHAR(255) UNIQUE,
  class VARCHAR(50),
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS enrollments (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  student_id UUID NOT NULL REFERENCES students(id) ON DELETE CASCADE,
  subject_id UUID NOT NULL REFERENCES subjects(id) ON DELETE CASCADE,
  enrollment_date TIMESTAMP DEFAULT NOW(),
  UNIQUE(student_id, subject_id)
);
```

Implement 2: Academic core tables

5.2.2 Timetable and Class Sessions

These tables separate fixed schedule definitions from runtime classroom sessions.

```sql
CREATE TABLE IF NOT EXISTS timetable (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  subject_id UUID NOT NULL REFERENCES subjects(id) ON DELETE CASCADE,
  teacher_id UUID NOT NULL REFERENCES teachers(id) ON DELETE CASCADE,
  room_id UUID NOT NULL REFERENCES rooms(id) ON DELETE CASCADE,
  day_of_week INT NOT NULL,
  start_time TIME NOT NULL,
  end_time TIME NOT NULL,
  expected_students INT DEFAULT 0,
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
);
```

```sql
CREATE TABLE IF NOT EXISTS class_sessions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  room_id UUID NOT NULL REFERENCES rooms(id) ON DELETE CASCADE,
  teacher_id UUID NOT NULL REFERENCES teachers(id),
  subject_id UUID NOT NULL REFERENCES subjects(id),
  timetable_id UUID REFERENCES timetable(id),
  mode VARCHAR(20) DEFAULT 'NORMAL',
  start_time TIMESTAMP DEFAULT NOW(),
  end_time TIMESTAMP,
  students_present JSONB DEFAULT '[]',
  final_performance_score FLOAT,
  final_risk_score FLOAT,
  status VARCHAR(20) DEFAULT 'ACTIVE',
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
);
```

Implement 3: Schedule and runtime session tables

Design Notes:

- class_sessions stores real-time mode and lifecycle status.
- students_present is a JSONB list to support fast runtime updates.

5.3 AI Monitoring and Analytics Tables

5.3.1 Behavior Classes, Logs, and Aggregates

These tables capture detections from AI inference and persist computed performance summaries.

```sql
CREATE TABLE IF NOT EXISTS behavior_classes (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  class_name VARCHAR(100) NOT NULL UNIQUE,
  actor_type VARCHAR(20) NOT NULL,
  description TEXT,
  is_active BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS behavior_logs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  session_id UUID NOT NULL REFERENCES class_sessions(id) ON DELETE CASCADE,
  actor_id UUID NOT NULL,
  actor_type VARCHAR(20) NOT NULL,
  behavior_class VARCHAR(100) NOT NULL,
  count INT DEFAULT 1,
  duration_seconds INT DEFAULT 0,
  detected_at TIMESTAMP DEFAULT NOW(),
  frame_snapshot BYTEA,
  yolo_confidence FLOAT DEFAULT 0.0,
  created_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS performance_aggregates (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  session_id UUID NOT NULL REFERENCES class_sessions(id) ON DELETE CASCADE,
  actor_id UUID NOT NULL,
  actor_type VARCHAR(20) NOT NULL,
  total_score FLOAT DEFAULT 0.0,
```

```
  behavior_breakdown JSONB DEFAULT '{}',
  calculated_at TIMESTAMP DEFAULT NOW(),
  UNIQUE(session_id, actor_id)
);
```

Implement 4: AI behavior and analytics tables

Design Notes:

- behavior_logs stores per-frame detections with confidence and optional snapshots.
- performance_aggregates supports dashboard-friendly precomputed scores.

5.4 Risk and Incident Management Tables

5.4.1 Risk Behaviors and Incidents

These tables implement cheat-risk taxonomy and incident tracking during testing mode.

```
CREATE TABLE IF NOT EXISTS risk_behaviors (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  behavior_name VARCHAR(100) NOT NULL UNIQUE,
  description TEXT,
  is_active BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS risk_incidents (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  session_id UUID NOT NULL REFERENCES class_sessions(id) ON DELETE CASCADE,
  student_id UUID NOT NULL REFERENCES students(id),
  risk_score FLOAT NOT NULL,
  risk_level VARCHAR(20) NOT NULL,
  triggered_behaviors JSONB NOT NULL,
  frame_snapshot BYTEA,
  flagged_at TIMESTAMP DEFAULT NOW(),
  reviewed BOOLEAN DEFAULT FALSE,
  reviewer_id UUID REFERENCES teachers(id),
  reviewer_notes VARCHAR(500),
  reviewed_at TIMESTAMP,
  created_at TIMESTAMP DEFAULT NOW()
);
```

Implement 5: Risk and incident tables

5.5 IoT Automation Tables

5.5.1 iot_rules and device_states

These tables manage policy-driven automation and real-time per-device status.

```
CREATE TABLE IF NOT EXISTS iot_rules (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  rule_name VARCHAR(255) NOT NULL,
  room_id UUID NOT NULL REFERENCES rooms(id) ON DELETE CASCADE,
  condition_type VARCHAR(50) NOT NULL,
  condition_params JSONB NOT NULL,
  actions JSONB NOT NULL,
  is_active BOOLEAN DEFAULT TRUE,
  priority INT DEFAULT 0,
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW(),
```

```sql
  last_triggered TIMESTAMP
);

CREATE TABLE IF NOT EXISTS device_states (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  room_id UUID NOT NULL REFERENCES rooms(id) ON DELETE CASCADE,
  device_id VARCHAR(255) NOT NULL,
  device_type VARCHAR(50) NOT NULL,
  status VARCHAR(20) NOT NULL DEFAULT 'OFF',
  last_toggled_by UUID REFERENCES teachers(id),
  manual_override BOOLEAN DEFAULT FALSE,
  override_until TIMESTAMP,
  last_updated TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW(),
  UNIQUE(room_id, device_id)
);
```

Implement 6: IoT automation tables

Key Features:

- JSONB condition_params and actions support extensible rule logic.
- Composite uniqueness on (room_id, device_id) ensures no duplicate device states.

5.6 Scoring Configuration Tables

5.6.1 performance_weights and risk_weights

These tables hold tunable parameters for learning score and risk score engines.

```sql
CREATE TABLE IF NOT EXISTS performance_weights (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  subject_id UUID REFERENCES subjects(id) ON DELETE SET NULL,
  behavior_name VARCHAR(100) NOT NULL,
  actor_type VARCHAR(20) NOT NULL,
  weight FLOAT NOT NULL,
  is_active BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW(),
  UNIQUE(subject_id, behavior_name, actor_type)
);

CREATE TABLE IF NOT EXISTS risk_weights (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  risk_behavior VARCHAR(100) NOT NULL UNIQUE,
  alpha_head_turn FLOAT DEFAULT 0.3,
  beta_talk FLOAT DEFAULT 0.5,
  gamma_device_use FLOAT DEFAULT 0.8,
  alert_threshold FLOAT DEFAULT 50.0,
  is_active BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
);
```

Implement 7: Scoring configuration tables

5.7 Security and Governance Tables

5.7.1 users Table

Stores authentication and role-based authorization data.

```sql
CREATE TABLE IF NOT EXISTS users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  username VARCHAR(255) NOT NULL UNIQUE,
  email VARCHAR(255) UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  role VARCHAR(50) NOT NULL DEFAULT 'LECTURER',
  is_active BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
);
```

Implement 8: users table structure

5.7.2 room_occupancy and audit_logs Tables

Support live occupancy snapshots and immutable operational traceability.

```sql
CREATE TABLE IF NOT EXISTS room_occupancy (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  room_id UUID NOT NULL REFERENCES rooms(id) ON DELETE CASCADE,
  occupancy_count INT DEFAULT 0,
  is_occupied BOOLEAN DEFAULT FALSE,
  last_detected TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW(),
  UNIQUE(room_id)
);

CREATE TABLE IF NOT EXISTS audit_logs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  entity_type VARCHAR(100) NOT NULL,
  entity_id UUID,
  action VARCHAR(50) NOT NULL,
  performed_by UUID REFERENCES users(id),
  changes JSONB,
  created_at TIMESTAMP DEFAULT NOW()
);
```

Implement 9: Occupancy and audit tables

5.8 Indexing Strategy

The schema includes explicit indexes to optimize joins, timeline queries, and dashboard filters:

- Hierarchy indexes: idx_floors_building_id, idx_rooms_floor_id
- Enrollment indexes: idx_enrollments_student_id, idx_enrollments_subject_id
- Session indexes: idx_class_sessions_room_id, idx_class_sessions_teacher_id, idx_class_sessions_start_time
- Behavior indexes: idx_behavior_logs_session_id, idx_behavior_logs_actor_id, idx_behavior_logs_detected_at
- Risk indexes: idx_risk_incidents_session_id, idx_risk_incidents_student_id
- IoT indexes: idx_device_states_room_id, idx_iot_rules_room_id
- Configuration and governance indexes: idx_performance_weights_subject_id, idx_room_occupancy_room_id, idx_audit_logs_entity_id, idx_audit_logs_created_at

5.9 Implementation Notes

- The script uses UUID primary keys with gen_random_uuid() defaults across all core tables.
- JSONB is used for flexible payloads: devices, students_present, behavior_breakdown, triggered_behaviors, and IoT rule actions.
- Seed data initializes behavior taxonomy, risk taxonomy, default performance weights, and default risk parameters.

6 Hardware Selection

To successfully demonstrate the Smart Scholar AIoT system, we selected a combination of microcontrollers, sensors, and simulating devices. A laptop acts as the central AI processing Gateway, while edge nodes handle data acquisition and actuation.

6.1 Processing Unit & Gateway

- **Laptop (Gateway):** Executes the AI models (FaceID, YOLO) using Python and manages MQTT communication with the edge nodes and the Adafruit IO server.
- **ESP32 / Yolo:Bit:** Serves as the central microcontroller for gathering sensor data and controlling output relays via WiFi.

6.2 Input Devices (Sensors & Cameras)

- **USB Webcam:** Used specifically for high-accuracy FaceID attendance at the classroom entrance. A direct connection to the Gateway prevents latency and network drops.
- **ESP32-CAM:** Positioned to monitor the classroom for occupancy counting and cheating detection.
- **DHT20 Sensor:** Chosen over the DHT11 for its higher precision and stable I2C communication to monitor temperature and humidity.

6.3 Output Devices (Actuators & Alerts)

- **4-Channel Relay Module (5V):** Controls the power state of the simulating devices.
- **Miniature DC Fans & LEDs:** Act as simulators for the classroom's actual HVAC and lighting systems.
- **16x2 I2C LCD:** Provides local, real-time display of the current system mode and environmental metrics.
- **5V Buzzer:** Emits audio alerts when specific conditions are met, such as the end of a timer or a cheating violation.

6.4 Hardware Summary Table

To provide a clear and concise overview of the system architecture, Table 23 summarizes the required devices, their quantities for the demonstration, and their designated installation locations.

Table 23: Hardware Equipment and Deployment Summary

No.	Device Name	Quantity	Installation Location
1	Laptop (AI Gateway)	1	Lecturer's Desk (Command Center)
2	ESP32 / Yolo:Bit	1	Lecturer's Desk / Central Control Box
3	USB Webcam (FaceID)	1	Entrance Zone (Wall, 1.5m - 1.6m height)
4	ESP32-CAM (Surveillance)	1	Ceiling Zone (Bird's-eye view)
5	DHT20 Sensor	1	Center Wall Zone (1.8m height)
6	4-Channel Relay Module	1	Central Control Box (Near ESP32)
7	Miniature DC Fans	2	Ceiling / Wall (Simulating HVAC)
8	Miniature LEDs	3	Ceiling (Simulating Zone Lighting)
9	16x2 I2C LCD	1	Lecturer's Desk
10	5V Buzzer	1	Lecturer's Desk

7 Device Control Logic

The system's actuators operate autonomously based on predefined logic rules, environmental data, and AI-driven inputs.

7.1 Lighting Control

- Lights are only operational during scheduled class hours.
- The system utilizes AI to detect student seating positions. It dynamically activates specific lighting zones based on occupancy, automatically turning off all lights if the room remains empty for 10 minutes.

7.2 HVAC (Fan and AC) Control

- If the room temperature exceeds 28°C and occupancy is detected, both the AC and fans are activated for rapid cooling.
- Once the temperature drops to 26°C, fans are turned off, and the AC maintains the ambient temperature.
- In moderate conditions (25°C - 27°C), only the fans are utilized to ensure air circulation while minimizing energy consumption.

7.3 Camera and Security Policy

- **Attendance Camera:** Operates only during the first 15 minutes of the session to record entry, minimizing CPU load thereafter.
- **Surveillance Camera:** In Learning Mode, it captures periodic frames (e.g., every 5 minutes) strictly for occupancy counting. In Testing Mode, it actively monitors for gaze deviation but does not record video. It only captures a single snapshot if a violation is detected, strictly adhering to privacy policies.

7.4 Audio and Visual Control

- In Learning Mode, the projector and speakers function normally.

- In Testing Mode, the system enters "Lockdown": audio output is muted, the projector switches to a countdown timer, and only the buzzer is authorized to emit short alerts for proctors if cheating is suspected.

8 Physical Setup and Deployment Layout

The strategic placement of IoT devices is crucial for accurate data collection and effective system response.

8.1 Entrance Zone

- **Device:** USB Webcam (Attendance).
- **Placement:** Mounted on the wall next to the door at an average eye level (1.5m - 1.6m).
- **Justification:** Provides a direct, frontal view of students entering, ensuring optimal lighting conditions from the hallway for the facial recognition model.

8.2 Center Wall Zone

- **Device:** DHT20 Temperature & Humidity Sensor.
- **Placement:** Mounted centrally on an interior wall, approximately 1.8m from the floor.
- **Justification:** Avoids direct sunlight from windows, cold drafts from the AC, and hallway air from the door, providing the most accurate representation of the room's ambient temperature.

8.3 Ceiling Zone

- **Device:** ESP32-CAM & Lighting System.
- **Placement:** Installed on the ceiling with a bird's-eye view angled towards the seating area.
- **Justification:** Prevents line-of-sight occlusion by students, allowing the AI to effectively count occupancy, map seating zones for localized lighting, and monitor a wide area for exam supervision.

8.4 Lecturer's Desk (Command Center)

- **Device:** LCD Display, Microphone, Buzzer, and Gateway Laptop.
- **Placement:** Positioned on the lecturer's podium.
- **Justification:** Centralizes control and monitoring, enabling the lecturer to instantly view environmental status, receive immediate buzzer alerts, and input voice commands without leaving the desk.

9 Dashboard

10 Architecture design

The system is deployed in a standard lecture hall. A central Gateway (Raspberry Pi or Laptop) processes AI tasks, while Yolo:Bit/ESP32 nodes handle sensor data and actuation. They communicate via MQTT to the Adafruit IO Cloud.

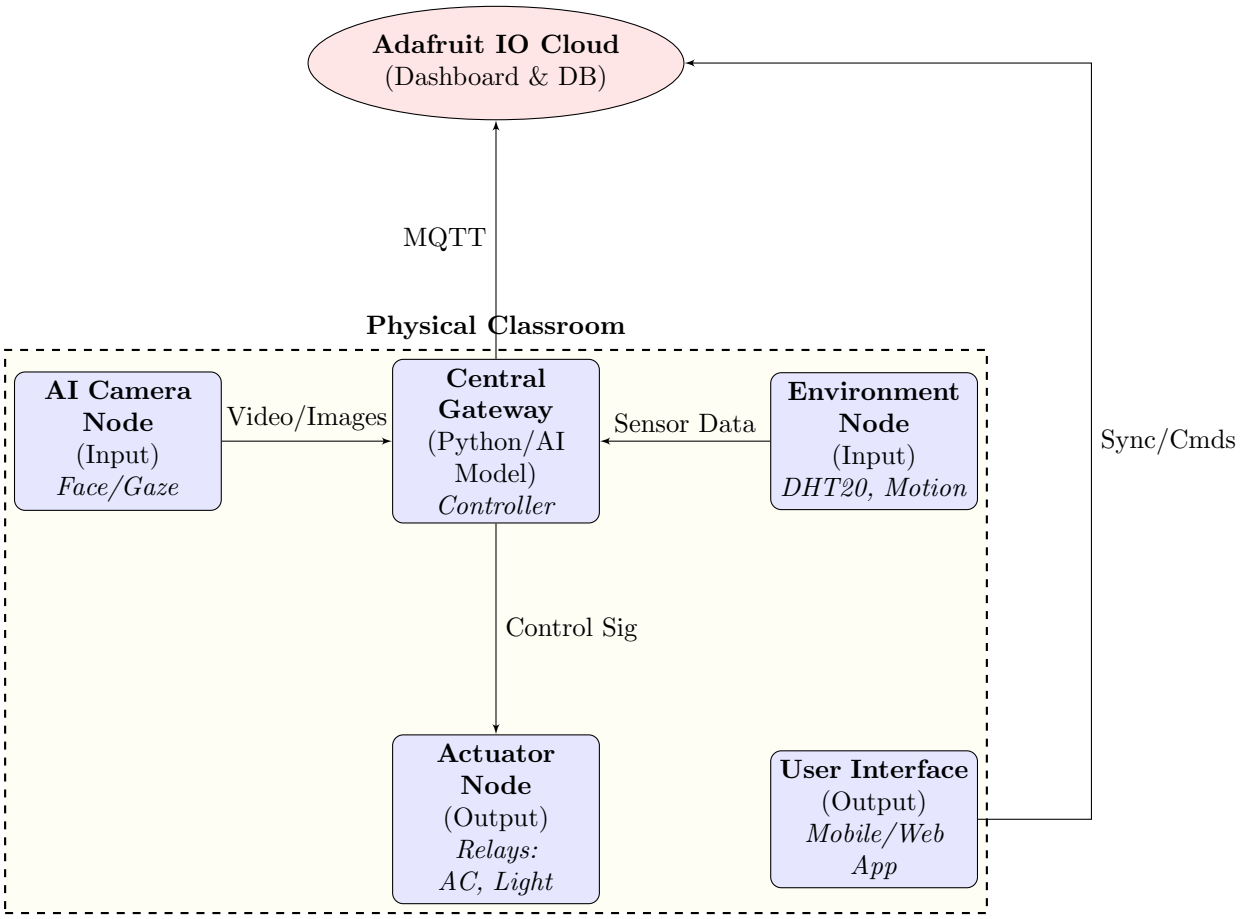


Figure 1: System Architecture in Real-World Context